chpt_6 - RNN 报告

姓名：王莹
学号：2054328

1.解释 RNN、LSTM、GRU 模型

1.1 RNN 模型

RNN（Recurrent Neural Network）是一类处理序列数据的神经网络。与普通前馈网络不同，RNN 在时刻 t 的输出不仅依赖当前输入 x_t，还依赖上一时刻的隐藏状态 h_{t-1}，因此可以建模上下文信息。

其基本形式可写为：

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h)$$

$$y_t = W_y h_t + b_y$$

其中，ϕ 常用 $\tanh$ 或 ReLU。RNN 的优点是结构简单，能够表达时序依赖；缺点是在长序列上容易出现梯度消失或梯度爆炸。

1.2 LSTM 模型

LSTM 是 RNN 的改进结构，核心是引入细胞状态 c_t 和门控机制（遗忘门、输入门、输出门），从而缓解普通 RNN 对长依赖建模困难的问题。

常见计算形式：

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \quad \tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

LSTM 的主要优势是能更稳定地保留长期信息。

1.3 GRU 模型

GRU（Gated Recurrent Unit）可以看作 LSTM 的简化版本，使用更新门和重置门来控制信息流，参数更少、训练更快。

其常见形式为：

$$z_t = \sigma(W_z[x_t, h_{t-1}] + b_z), \quad r_t = \sigma(W_r[x_t, h_{t-1}] + b_r)$$

$$\tilde{h}_t = \tanh(W_h[x_t, r_t \odot h_{t-1}] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU 在不少任务上效果接近 LSTM，但结构更轻量。

2.叙述诗歌生成过程

本实验使用 TensorFlow 版本实现中文诗歌生成，整体流程如下：

1. 读取数据集 `poems.txt` ，为每条样本加上起始标记 `bos` 和结束标记 `eos` 。
2. 构建词表 `word2id` 、 `id2word` ，将汉字序列转成 id 序列。
3. 构建 `tf.data` 数据管道，输入为 `x[:, :-1]` ，标签为 `x[:, 1:]` 。
4. 模型结构采用：Embedding(64) -> SimpleRNNCell(128) -> Dense(vocab_size)。
5. 训练时使用交叉熵损失并按有效序列长度做平均。
6. 生成时给定 `begin_word` ，逐步预测下一个 token，遇到 `eos` 或达到最大长度停止。

我在代码中做了与当前环境相关的一些必要兼容处理：

1. 数据路径改为 `./poems.txt` ，避免文件找不到。
2. `Embedding` 删除 `batch_input_shape` 参数（适配 Keras 3）。
3. 去除部分 `@tf.function` ，否则运行时梯度为 `None` 。

本实验中使用的生成函数如下：

```python
def gen_sentence(begin_word, max_len=50, min_len=12):
    state = [tf.random.normal(shape=(1, 128), stddev=0.5)]
    cur_token = tf.constant([word2id[begin_word]], dtype=tf.int32)
    collect = [word2id[begin_word]]

    for _ in range(max_len):
        cur_token, state = model.get_next_token(cur_token, state)
        tid = int(cur_token.numpy()[0])

        if tid == word2id['eos'] and len(collect) < min_len:
            continue
        if tid == word2id['eos']:
            break

        collect.append(tid)

    return [id2word[t] for t in collect]
```

3.指定开头词生成结果

按题目要求，使用以下开头词生成：

- 日
- 红
- 山
- 夜
- 湖
- 海
- 月

本次实验生成结果（某次训练结果）如下：

1. 日：日暮雨沾巾看看花。得不知何处处，不知何处是谁家。
2. 红：红蕖。玉双旌下马蹄，一枝红叶不如丝。
3. 山：山茫茫。尽不知此，何人不可知。
4. 夜：夜夜江上，山山一夜深。花开户里，风雨入山山。
5. 湖：湖畔忆往来逢此日期。来不得无人事，不得春风一夜来。
6. 海：海上城边。云山水上，风雨夜中秋。

7. 月：月下门前路，不见春风一夜来。

3.1 实验过程截图

1. 训练日志截图：

```python
optimizer = optimizers.Adam(0.0005)
train_ds, word2id, id2word = poem_dataset()
model = myRNNModel(word2id)

for epoch in range(10):
    loss = train(epoch, model, optimizer, train_ds)
```

[24] ✓ 59m 29.2s Python

```
epoch 0 : loss 8.820923
epoch 1 : loss 6.5767045
epoch 2 : loss 6.0381446
epoch 3 : loss 5.8409276
epoch 4 : loss 5.6209474
epoch 5 : loss 5.485834
epoch 6 : loss 5.4426146
epoch 7 : loss 5.370477
epoch 8 : loss 5.2919574
epoch 9 : loss 5.263072
```

```python
optimizer.learning_rate.assign(1e-4)
for epoch in range(30, 50):
    loss = train(epoch, model, optimizer, train_ds)
    print("continue epoch", epoch, ": loss", float(loss.numpy()))
```

[31] ✓ 112m 23.5s Python

```
epoch 10 : loss 5.132954
continue epoch 10 : loss 4.583153247833252
epoch 11 : loss 5.148712
continue epoch 11 : loss 4.852517127990723
epoch 12 : loss 5.106836
continue epoch 12 : loss 5.247089862823486
epoch 13 : loss 5.053562
continue epoch 13 : loss 4.755221843719482
epoch 14 : loss 5.071986
continue epoch 14 : loss 5.186461448669434
epoch 15 : loss 5.12137
continue epoch 15 : loss 4.727019786834717
epoch 16 : loss 5.02956
continue epoch 16 : loss 5.207489013671875
epoch 17 : loss 5.097562
continue epoch 17 : loss 4.90928316116333
epoch 18 : loss 4.9870143
continue epoch 18 : loss 5.096435546875
epoch 19 : loss 5.0519347
continue epoch 19 : loss 4.899768352508545
epoch 20 : loss 4.990797
continue epoch 20 : loss 4.859710693359375
epoch 21 : loss 5.0363483
continue epoch 21 : loss 4.72069787979126
epoch 22 : loss 4.983184
...
epoch 28 : loss 4.888927
continue epoch 28 : loss 5.241137981414795
epoch 29 : loss 4.866466
continue epoch 29 : loss 4.9487786293029785
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
optimizer.learning_rate.assign(1e-4)
for epoch in range(30, 50):
    loss = train(epoch, model, optimizer, train_ds)
    print("continue epoch", epoch, ": loss", float(loss.numpy()))
```

[37] ✓ 107m 7.1s Python

```
epoch 30 : loss 4.9276733
continue epoch 30 : loss 4.788247585296631
epoch 31 : loss 4.855887
continue epoch 31 : loss 4.99505615234375
epoch 32 : loss 4.8223968
continue epoch 32 : loss 4.470088481903076
epoch 33 : loss 4.835307
continue epoch 33 : loss 5.041310787200928
epoch 34 : loss 4.922865
continue epoch 34 : loss 4.990031719207764
epoch 35 : loss 4.7584634
continue epoch 35 : loss 4.523951053619385
epoch 36 : loss 4.8410244
continue epoch 36 : loss 5.063176155090332
epoch 37 : loss 4.866434
continue epoch 37 : loss 4.705322265625
epoch 38 : loss 4.8210435
continue epoch 38 : loss 5.126391410827637
epoch 39 : loss 4.7702236
continue epoch 39 : loss 4.836478233337402
epoch 40 : loss 4.823235
continue epoch 40 : loss 4.768317699432373
epoch 41 : loss 4.6504793
continue epoch 41 : loss 4.905481815338135
epoch 42 : loss 4.775351
...
epoch 48 : loss 4.83563
continue epoch 48 : loss 4.828632831573486
epoch 49 : loss 4.782505
continue epoch 49 : loss 4.8183746337890625
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

2. 7 个开头词生成结果截图：

```python
for w in ['日','红','山','夜','湖','海','月']:
    print('【开头】',w)
    print(''.join(gen_sentence(w)))
    print('-'*30)
```

[30] ✓ 0.2s Python

```
【开头】 日
日斜。
------------------------------
【开头】 红
红叶满楼花落月寒。
------------------------------
【开头】 山
山畔岸滨黏蓉畔柳鸪湲畔寞惆怅忆脁鹃递惆跎惆悴寞寞赊惆怅惆跎惆悴寞鹃递啻峡畔扉递递惆跎惆跎惆悴寞寞觅寞寞
------------------------------
【开头】 夜
夜斜塘月落花，一枝花。
------------------------------
【开头】 湖
湖上月，不见天，一年年。
------------------------------
【开头】 海
海上楼中日，风光不可知。
------------------------------
【开头】 月
月。
------------------------------
```

```python
for w in ['日','红','山','夜','湖','海','月']:
    print('【开头】',w)
    print(''.join(gen_sentence(w)))
    print('-'*30)
```

[36] ✓ 0.1s Python

```
【开头】 日
日曛，只是山中去，何人不得归。
------------------------------
【开头】 红
红叶滴帘栊。
------------------------------
【开头】 山
山水。
------------------------------
【开头】 夜
夜泊猿啼。
------------------------------
【开头】 湖
湖南望望，风景不知君。
------------------------------
【开头】 海
海滨肃穆穆穆，不知何处事，不知何处为人。
------------------------------
【开头】 月
月如何处是知君。
------------------------------
```

```python
for w in ['日','红','山','夜','湖','海','月']:
    print('【开头】',w)
    print(''.join(gen_sentence(w)))
    print('-'*30)
```

[42] ✓ 0.1s Python

```
【开头】 日
日斜，一声还不得，不知何处是何人。
------------------------------
【开头】 红
红蕖袅袅袅袅袅。
------------------------------
【开头】 山
山水滴帘开户牖，风吹落叶满庭花。
------------------------------
【开头】 夜
夜莺啼猿啼处处，不见山中月。
------------------------------
【开头】 湖
湖上有春风雨，春风吹落花。
------------------------------
【开头】 海
海滨》）
------------------------------
【开头】 月
月霏霏霭，风吹落。
------------------------------
```

4.总结

本次实验完成了代码运行与结果分析。

1. 在模型认知方面，理解了 RNN、LSTM、GRU 的结构差异及适用场景。
2. 在实现方面，完成了 TensorFlow 诗歌生成代码的补全与环境适配。
3. 在结果方面，成功实现指定 7 个开头词的诗句生成。

从训练现象看，loss 从初期约 8.82 下降到约 4.8 左右，说明模型已经学习到一定序列模式；但输出中仍有重复词和异常字符，可以看出 SimpleRNN 在复杂文本生成任务中的局限。后续若采用 LSTM/GRU 或其它改进策略或许生成质量还有更大提升空间。